

A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is a light green color. They are positioned diagonally, with the blue one partially covering the green one.

AI Agents Frameworks

Filipe de Barros Carneiro Silva



Frameworks vistos

CrewAI

LangGraph

SmolAgents

Agno

AutoGen

AtomicAgents

Semantic Kernel: Parece ser focado para empresas e não um sistema de agentes e tem pouca gente usando

Mastra: JavaScript

PydanticAI: Pouco explorado porque os outros já estavam bem explorados



AutoGen



AG

AutoGen é um framework da Microsoft com foco em alta agencia dos agentes.
Os agentes são capazes de se auto organizar para resolver os problemas.

Pontos positivos:

- Alta autonomia
- Flexível

Pontos negativos:

- Difícil de controlar
- Difícil debug



CrewAi



CrewAI é um framework de alta abstração que permite criar times de agentes de maneira fácil e intuitiva como se estivesse criando personagens.

Pontos positivos:

- Altamente intuitivo
- Estrutura de organização clara
- Foco em multi agentes
- Boa interface

Pontos negativos:

- Baixa agência
- Debug pode ser difícil
- Possui uma estrutura bem definida de times
- Não tem muito controle

AtomicAgents



É um framework que utiliza uma abordagem de “blocos de lego” para montar os agentes. Você constrói agents juntando pequenas partes para formar o todo.

Pontos positivos:

- Alto controle
- Modularidade e reutilização

Pontos negativos:

- Orquestração totalmente manual
- Agentes são limitados ao que você define para eles

Ver exemplos no repositório do git

SmolAgents



SmolAgents é um framework de baixa abstração que dá total liberdade ao programador em como montar seu sistema. Possui alta agência, ou seja, os agentes possuem mais liberdade para pensarem como quiserem. Seu diferencial é que ele encaminha dados usando códigos em python invés de JSON.

Pontos positivos:

- Leve eficiente
- Prototipagem rápida
- Desenvolvedor tem total controle

Pontos negativos:

- Orquestração manual
- Não possui muitas coisas prontas, logo você deverá fazer

Ver exemplo de blog writer

Instruction: Determine the most cost-effective country to purchase the smartphone model "CodeAct 1". The countries to consider are the USA, Japan, Germany, and India.

Available APIs

[1] lookup_rates(country: str) -> (float, float)

[2] convert_and_tax(price: float, exchange_rate: float, tax_rate: float) -> float

[3] estimate_final_price(converted_price: float, shipping_cost: float) -> float

[4] lookup_phone_price(model: str, country: str) -> float

[5] estimate_shipping_cost(destination_country: str) -> float

LLM Agent using [Text/JSON] as Action

 **Think** I should calculate the phone price in USD for each country, then find the most cost-effective country.

 **Action** **Text:** lookup_rates, Germany
JSON: {"tool": "lookup_rates", "country": "Germany"}

 **Environment** 1.1, 0.19

 **Action** **Text:** lookup_phone_price, CodeAct 1, Germany
JSON: {"tool": "lookup_phone_price", "model": "CodeAct 1", "country": "Germany"}

 **Environment** 700

 **Action** **Text:** convert_and_tax, 700, 1.1, 0.19
JSON: {"tool": "convert_and_tax", "price": 700, "exchange_rate": 1.1, "tax_rate": 0.19}

 **Environment** 916.3

[... interactions omitted (look up shipping cost and calculate final price) ...]

 **Action** **Text:** lookup_rates, Japan
JSON: {"tool": "lookup_rates", "country": "Japan"}

[... interactions omitted (calculate final price for all other countries) ...]

 **Response** The most cost-effective country to purchase the smartphone model is Japan with price 904.00 in USD.

Fewer Actions Required!

CodeAct: LLM Agent using [Code] as Action

 **Think** I should calculate the phone price in USD for each country, then find the most cost-effective country.

 **Action**

```
countries = ['USA', 'Japan', 'Germany', 'India']
final_prices = {}

for country in countries:
    exchange_rate, tax_rate = lookup_rates(country)
    local_price = lookup_phone_price("xAct 1", country)
    converted_price = convert_and_tax(
        local_price, exchange_rate, tax_rate
    )
    shipping_cost = estimate_shipping_cost(country)
    final_price = estimate_final_price(converted_price, shipping_cost)
    final_prices[country] = final_price

most_cost_effective_country = min(final_prices, key=final_prices.get)
most_cost_effective_price = final_prices[most_cost_effective_country]
print(most_cost_effective_country, most_cost_effective_price)
```

Control & Data Flow of Code
Simplifies Complex Operations

 **Environment** 1.1, 0.19

 **Response** The most cost-effective country to purchase the smartphone model is Japan with price 904.00 in USD.

Re-use 'min' Function from Existing
Software Infrastructures (Python library)



Agno



Agno é um framework open source, de alta nível que permite criar agentes e times de maneira simples e rápida. É capaz de criar times de times o que pode ser algo útil na organização de um sistema complexo. Também é considerado um dos frameworks mais otimizados e eficientes.

Pontos positivos:

- Fácil de usar
- Multimodal
- Eficiente
- Possui muita coisa já implementada

Pontos negativos:

- Baixa agência

Ver exemplo de `movie_recommendation`



LangGraph



LangGraph

LangGraph é um framework originário do LangChain que permite a construção de redes de agentes extremamente controladas devido a uma organização baseada em grafos.

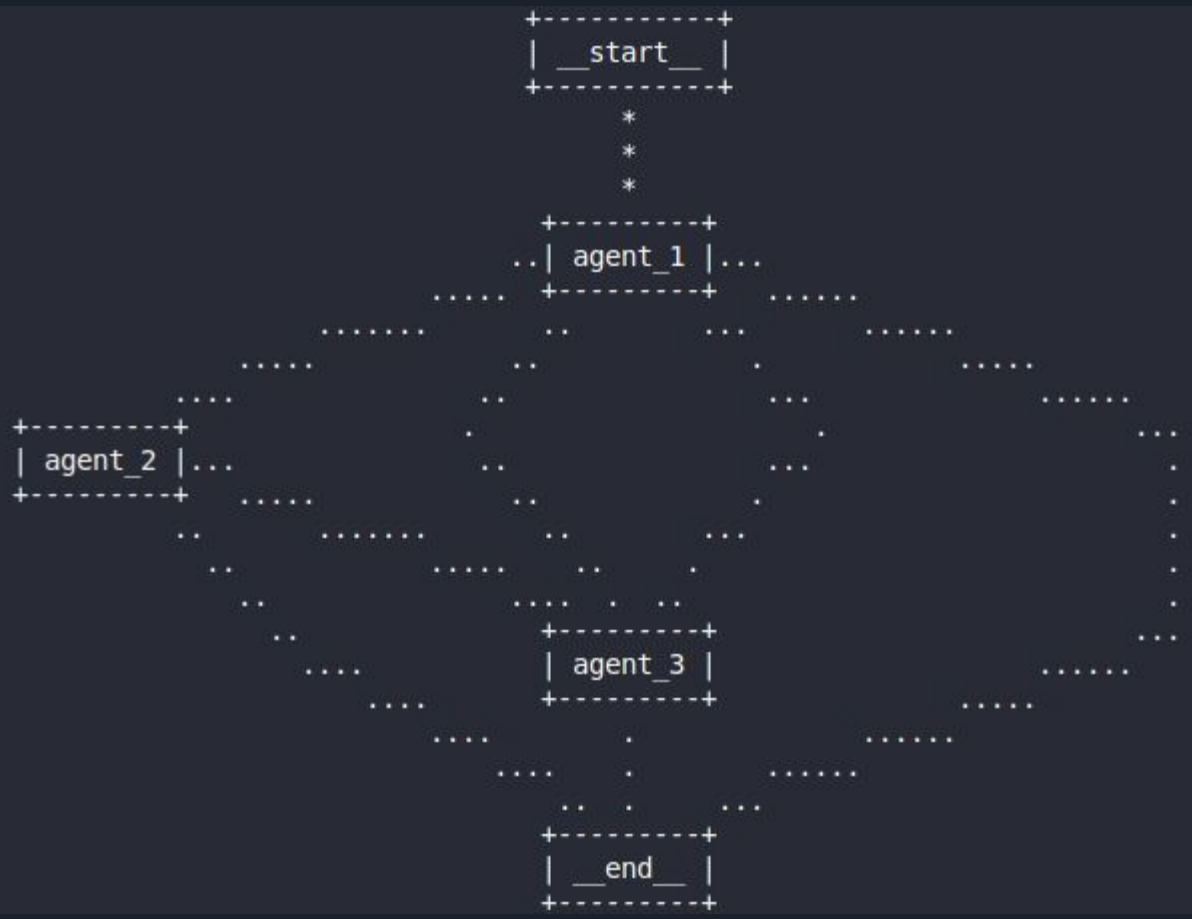
Pontos positivos:

- Controle total sobre o fluxo de execução dos agentes
- MultiModal
- Integração com LangChain

Pontos negativos:

- Pode ser um pouco difícil entender o funcionamento no início
- Em sistemas complexos, o debug pode ser complicado
- Framework técnico que precisa de muito código

Ver qualquer um dos exemplos no git



Critério	Análise Detalhada
Framework: CrewAI	
Gratuidade	Sim
Usabilidade	Alta, a definição dos agentes é bem simples e não tem muita complexidade.
Flexibilidade	Alta, os prompts podem ser alterados de maneira simples definindo as características dos agentes.
Agnosticismo	Sim.
Nível de abstração	Muito alta, a definição dos agentes é simples e não permite alterações de baixo nível.
Modelo de ação	Ferramentas.
Agência	Baixa, é estritamente ligado ao formato de times e os agentes não fogem do que é definido a ele.
Desenvolvimento ativo	Sim, mantido por uma startup brasileira e o último commit foi no dia 02/10/2025.
Framework: LangGraph	
Gratuidade	Sim
Usabilidade	Média, a abordagem de grafos pode ser complexa no início.
Flexibilidade	Alta, você altera os prompts diretamente.
Agnosticismo	Sim.
Nível de abstração	Baixa, é necessário definir cada aresta do grafo para o sistema funcionar.
Modelo de ação	Ferramentas.
Agência	Baixa, o controle de fluxo dos agentes cabe ao desenvolvedor.
Desenvolvimento ativo	Sim, mantido pela mesma organização do LangChain e o último commit foi dia 03/10/2025.
Framework: AtomicAgents	
Gratuidade	Sim
Usabilidade	Baixa, é necessário que você monte cada componente do agente.
Flexibilidade	Alta, cada componente do agente tem uma função e tarefa bem definida.
Agnosticismo	Sim.
Nível de abstração	Baixa, sistema modular em que é necessário definir cada componente do agente.
Modelo de ação	Ferramentas.
Agência	Baixa, cada componente do agente segue o padrão definido a ele.
Desenvolvimento ativo	Sim, é mantido por um grupo de pessoas e o último commit foi dia 25/09/2025.

Critério	Análise Detalhada
Gratuidade	Sim
Usabilidade	Alta, fornece muitas ferramentas e funcionalidade prontas.
Flexibilidade	Alta, o prompt é facilmente alterável.
Agnosticismo	Sim.
Nível de abstração	Alta, é simples definir agentes e times.
Modelo de ação	Ferramentas.
Agência	Baixa, os agentes seguem aquilo que foi definido no papel deles.
Desenvolvimento ativo	Sim, o desenvolvimento é mantido pela empresa de mesmo nome e o último commit foi dia 05/10/2025.
Framework: AutoGen	
Gratuidade	Sim
Usabilidade	Média, é necessário técnica para controlar os agentes livres.
Flexibilidade	Alta, A personalização do prompt é feita diretamente no agente.
Agnosticismo	Sim.
Nível de abstração	Alta, os agentes são definidos de maneira simples, sem precisar de alterações de baixo nível.
Modelo de ação	Ferramentas.
Agência	Alta, agentes se auto organizam e decidem como interagir.
Desenvolvimento ativo	Sim, é mantido pela Microsoft e o último commit foi dia 03/10/2025.
Framework: SmolAgents	
Gratuidade	Sim
Usabilidade	Baixa, devido a forma como os agentes geram os códigos em python, é necessário um grande controle técnico.
Flexibilidade	Alta, é possível alterar prompts diretamente no agente.
Agnosticismo	Sim.
Nível de abstração	Baixa, requer um nível técnico maior para a definição dos agentes e ferramentas.
Modelo de ação	Ferramentas e geração de código.
Agência	Alta, os agentes "pensam em código" e decidem livremente sobre chamar ferramentas ou outros agentes, oferecendo muita autonomia.
Desenvolvimento ativo	Sim, é mantido pela empresa Hugging Face e o último commit foi feito dia 28/10/2025.